Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Sorting Algorithms

Prof. Reza Skandari

Today

1. Recap
2. Sorting algorithms:
    - Selection sort
    - Merge sort

Test your instincts

Poll time! Open menti.com and vote on each question.

What is the output?

```
a = 'hiphop'
b = a[2]
c = a[:len(a) - 1]
print(b, c)
```

A. `i hipho`

B. `p hipho`

C. `i hiph`

D. `p hiph`

E. I don't know

What is the output?

```
a = 'hip'
for i in range(1, len(a)):
    print(a[i] + a[i - 1])
```

A. An error

B. `hp, ih, pi`

C. `ih, pi`

D. `hi, ip`

E. I don't know

This algorithm is...?

```
def fun_function(s):
    # s is a list of length n
    new_list = []
    for v1 in s:
        for v2 in s:
            new_list.append(v1 + v2)
    return new_list
```

A. Constant time - $O(1)$

B. Linear time - $O(n)$

C. Quadratic time - $O(n^2)$

D. None of the above

E. I don't know

Which of these reverses the string?

```
s = 'hiphop'

new_s = s[0:len(s):1]                    # A

new_s = s[::-1]                          # B

new_s = ''                               # C
for char in s:
    new_s = char + new_s

new_s = ''                               # D
for i in range(len(s)):
    new_s += s[len(s) - i - 1]
```

A. B. C. D. E. More than one of the above

Asymptotic analysis

1- measure number of basic operations as a function of input size

2- focus on worst-case analysis

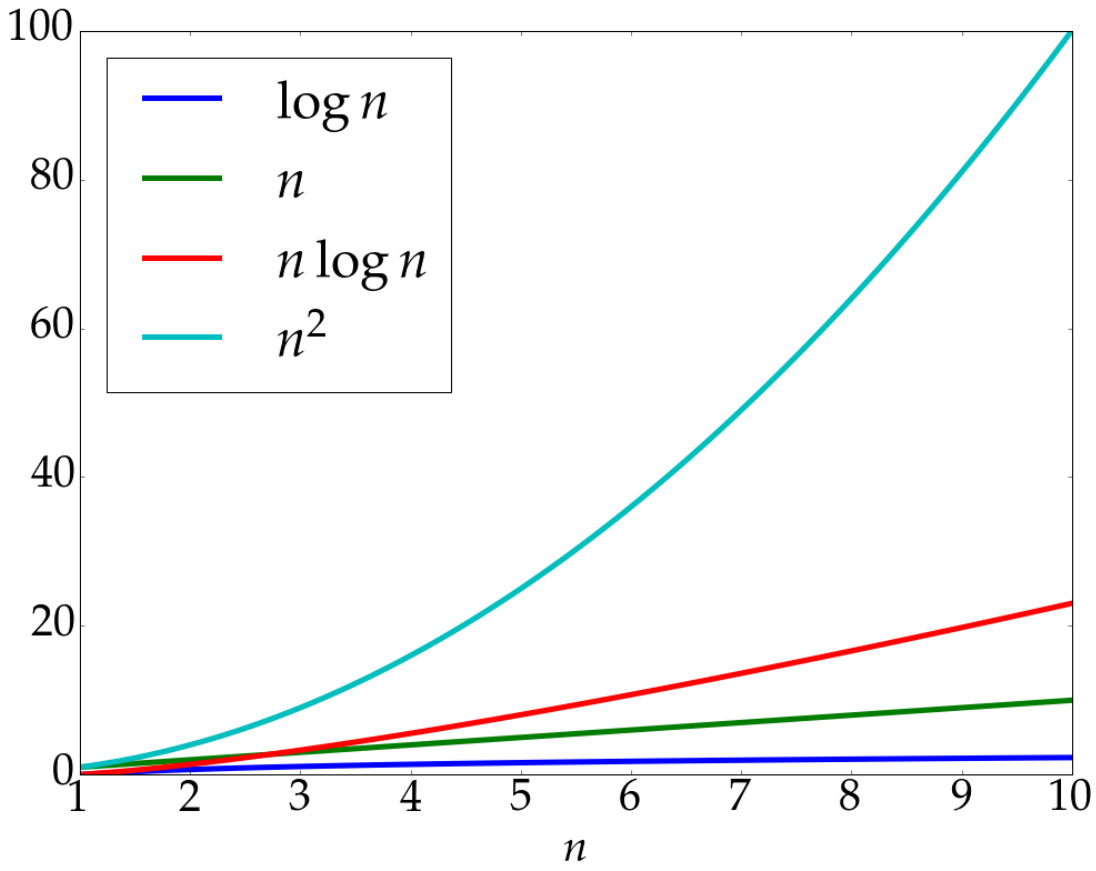
3- ignore constant factors and lower-order terms

4- only care about large inputs

Formal way to describe this approach:

- Big-O notation: upper bound on worst-case running time

Growth rates



Big O: for **large enough inputs**, an $O(n)$ algorithm will be slower than $O(\log(n))$

Basic operations

Operations that a computer can perform "quickly" (constant time $O(1)$ for any input):

- Arithmetic operations
- Comparisons
- Variable assignment
- Memory access

What if the data structure is more complicated?

- For example, if `L` is a list: `L.append()`, `L[5]`?
- Are these basic constant-time operations?
- Wait for it... assume for now that list operations are constant time

Complexity classes

Fast algorithm: worst-case running time grows slowly with input size

- $O(1)$: constant running time (basic operations)
- $O(\log n)$: logarithmic running time (binary search)
- $O(n)$: linear running time (linear search)
- $O(n \log n)$: log-linear running time (??)
- $O(n^c)$: polynomial running time (??)
- $O(c^n)$: exponential running time (??)

Sorting algorithms

So if we have an unsorted list, should we sort it first?

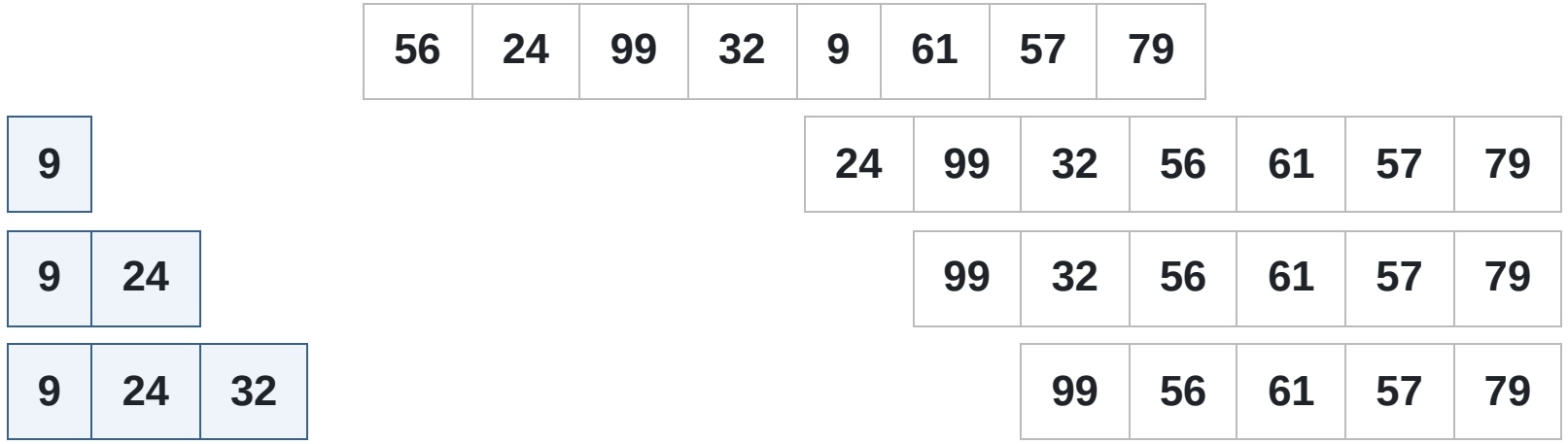
- Suppose complexity $O(sort(n))$
- Is it less work to sort and then do binary search than to do linear search?
- Is $sort(n) + \log(n) < n$?
- No...

But what if we need to search repeatedly, say k times?

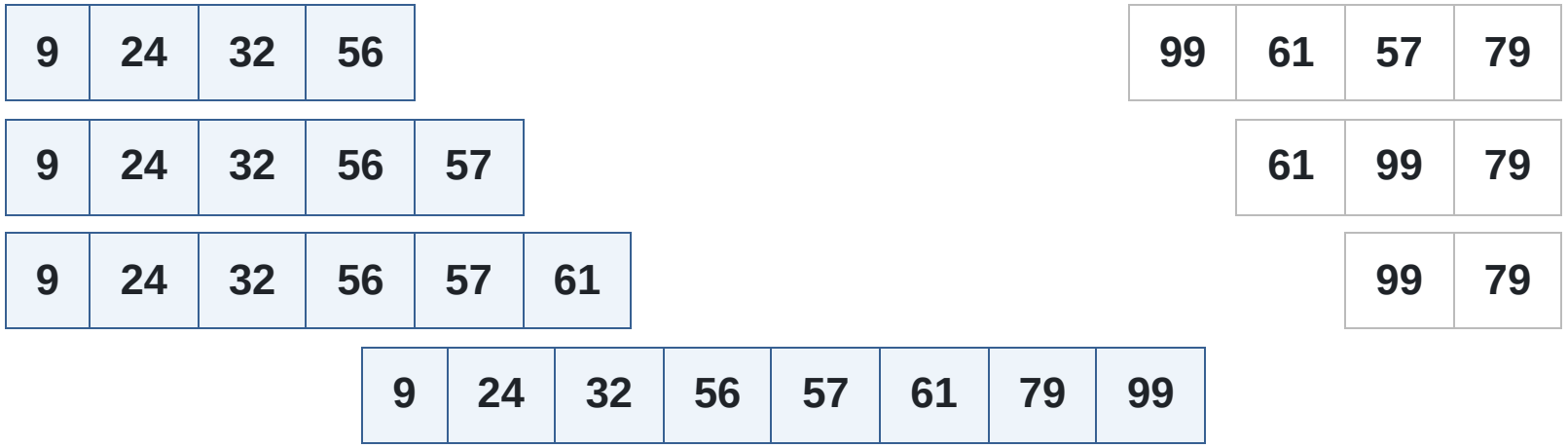
- Is $sort(n) + k\log(n) < kn$?
- Depends on k...

How would you intuitively sort a list?

Find the smallest item, swap it to the front. Repeat with the remaining unsorted items.



How would you intuitively sort a list?



In words: Find smallest item and move to front (swap with first unsorted item). Repeat with remaining unsorted items.

Selection sort algorithm

Selection sort list `L` of length `n`:

- Repeat `n` times:
  - Find smallest unsorted element
  - Swap its position with the first unsorted element

```
def selection_sort(L):
    M = L[:]  # make a copy to preserve original list
    n = len(M)
    for index in range(n):
        min_index = find_min_index(M, index)  # index with smallest element
        M[index], M[min_index] = M[min_index], M[index]  # swap positions
    return M
```

Selection sort complexity

Correctness (for those into math): can be proved by induction

Complexity:

- $O(n)$ passes of main loop
- Each pass: search for the smallest element in $O(n)$
- Total $O(n^2)$

Can we do better?

- Yes! **Merge sort** is $O(n \log n)$, but you can't do any better than that

To talk about merge sort, we first need to understand **recursion**.

Example: factorial

The factorial of n is the product of integers $1, ..., n$.

- As a function: $\text{fact}(n) = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$
- By convention, $\text{fact}(0) = 1$

```
def fact(n):
    result = 1
    for i in range(1, n+1):
        result = result * i
    return result

print(fact(4))
```

Visualize this: cscircles.cemc.uwaterloo.ca/visualize

But we can also write the factorial **recursively**:

$$\text{fact}(n) = 1 \text{ for } n = 0, \qquad \text{fact}(n) = n \times \text{fact}(n-1) \text{ for } n > 0$$

Factorial by recursion

We can also write the factorial as follows:

$$\text{fact}(n) = 1 \text{ for } n = 0, \qquad \text{fact}(n) = n \times \text{fact}(n-1) \text{ for } n > 0$$

Factorial can be expressed as a smaller version of itself:

```
def fact_rec(n):
    if n <= 1:
        return 1
    return n * fact_rec(n-1)

print(fact_rec(4))
```

This is called **recursion**:

- Function calls itself (until base case is reached)
- Can make some problems easier to define $\rightarrow$ merge sort!

Tracing `fact_rec(5)`

Going down: each call unwinds into a smaller one, until the base case is reached

```
fact_rec(5) = 5 * fact_rec(4)   since 5 > 1
fact_rec(4) = 4 * fact_rec(3)   since 4 > 1
fact_rec(3) = 3 * fact_rec(2)   since 3 > 1
fact_rec(2) = 2 * fact_rec(1)   since 2 > 1
fact_rec(1) = 1                 base case
```

Rolling back up: each waiting call resumes once the smaller one finishes

```
fact_rec(2) = 2 * 1  = 2
fact_rec(3) = 3 * 2  = 6
fact_rec(4) = 4 * 6  = 24
fact_rec(5) = 5 * 24 = 120
```

Divide and conquer

Three components to look for in a recursive algorithm:

- **Base case**: the smallest subproblem, small enough to solve directly
- **Divide**: break the problem into smaller subproblem(s) of the same type
- **Combine**: use the subproblem solution(s) to build the solution to the original problem

Example: factorial

- Base case: $\mathrm{fact}(1) = 1$
- Divide: $\mathrm{fact}(n)$ needs $\mathrm{fact}(n-1)$
- Combine: multiply n by the result of $\mathrm{fact}(n-1)$

Divide and conquer: fast exponentiation

Computing x^n:

- Naive: multiply x by itself n times $\rightarrow O(n)$

Divide and conquer:

- Base case: $x^0 = 1$
- Divide: x^n needs just one subproblem, $x^{n//2}$ (`//` is integer division, rounds down)
- Combine: square the result ($\times x$ extra if n is odd)

Examples:

- n even: $5^{50} = 5^{25} \times 5^{25}$
- n odd: $19^{39} = 19^{19} \times 19^{19} \times 19$

Fast exponentiation: code

```
def power(x, n):
    if n == 0:
        return 1
    half = power(x, n // 2)
    if n % 2 == 0:
        return half * half
    return half * half * x
```

Complexity: n is halved each call $\rightarrow O(\log n)$ multiplications, instead of $O(n)$

Merge sort as divide and conquer

- Base case: if list length $n < 2$, the list is sorted
- Divide: if list length $n \geq 2$:
  - split into two lists
  - **recursively merge sort each** to produce two sorted lists
- Combine: merge the two sorted lists into one

Combining two sorted halves

We split the original list into two halves:

`[56, 24, 99, 32, 9, 61, 57, 79]`

- left = `[56, 24, 99, 32]`
- right = `[9, 61, 57, 79]`

Recursively sorting each half gives us:

- left = `[24, 32, 56, 99]`
- right = `[9, 57, 61, 79]`

Now we need to **combine** them back into one sorted list.

Merging two sorted lists into one

Merge(left, right):

- Look at the front (unused) item of each list
- Copy the smaller of the two to `z`, and move past it in its list
- Repeat until one list runs out
- Copy whatever remains of the other list onto the end of `z`

Big picture: since both `left` and `right` are already sorted, the overall smallest remaining item is always at the front of one of them, so we never have to look any further than that.

Merging two sorted lists: walkthrough

left = `[24, 32, 56, 99]` right = `[9, 57, 61, 79]`

Copy the smaller item to `z`, advancing whichever index (`i1` or `i2`) we copied from:

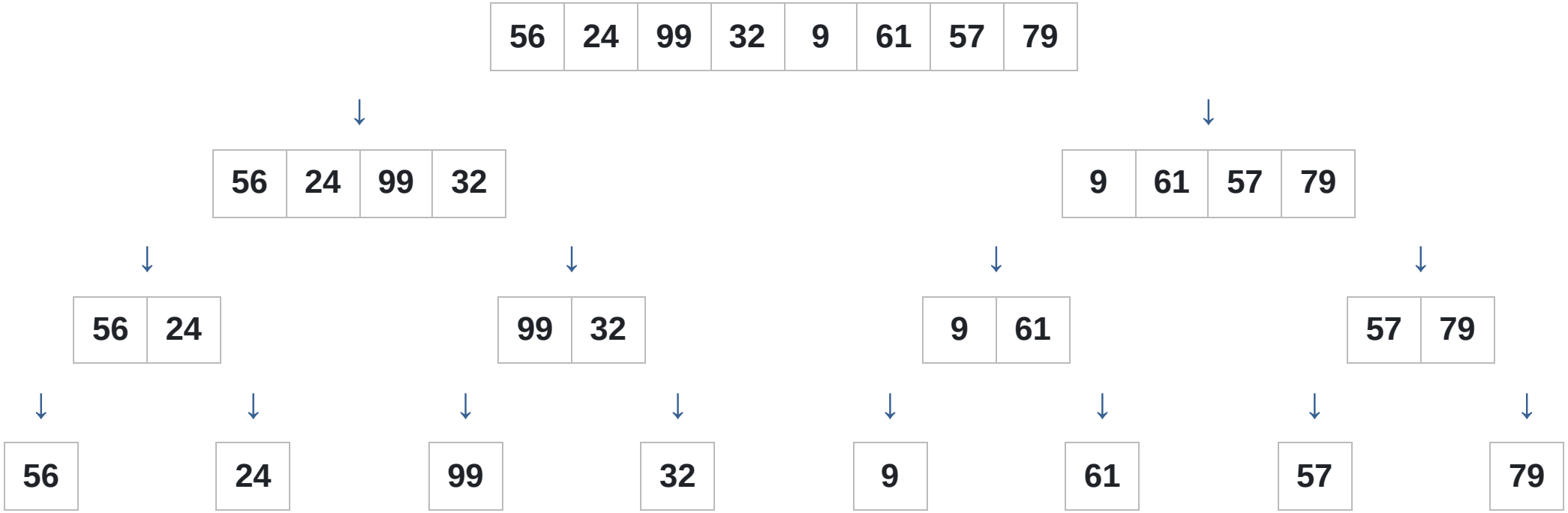
Step	i1	i2	z
start	0	0	`[]`
copy right[0]=9	0	1	`[9]`
copy left[0]=24	1	1	`[9, 24]`
copy left[1]=32	2	1	`[9, 24, 32]`
copy left[2]=56	3	1	`[9, 24, 32, 56]`
copy right[1]=57	3	2	`[9, 24, 32, 56, 57]`
copy right[2]=61	3	3	`[9, 24, 32, 56, 57, 61]`
copy right[3]=79	3	4	`[9, 24, 32, 56, 57, 61, 79]`
copy left[3]=99	4	4	`[9, 24, 32, 56, 57, 61, 79, 99]`

Merging: complexity

What is the complexity of this operation?

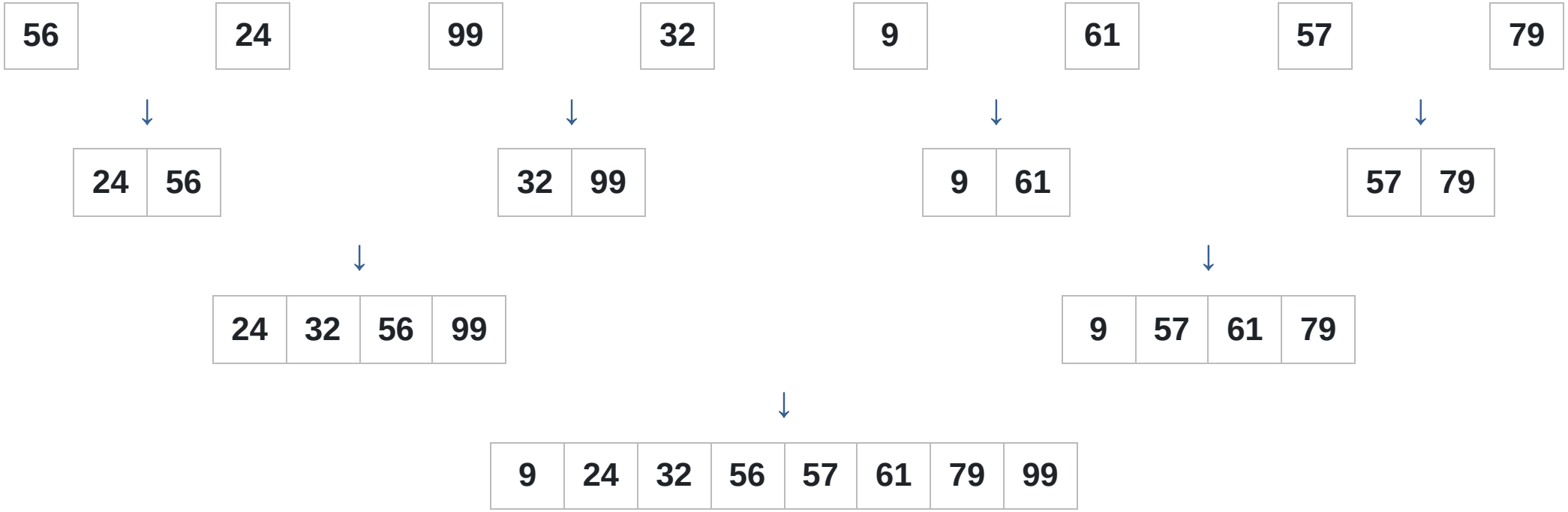
- Lengths of lists are n_1 and n_2
- Two lists of lengths n_1 and n_2: $O(n_1 + n_2)$ copy operations (need to copy each item)
- We also compare values, but no more comparisons than copy operations
  - Worst case: $n_1 + n_2 - 1$ comparisons, once one list empties, the rest of the other is copied for free
  - This worst case happens when the items from the two lists are perfectly interleaved (e.g. small, big, small, big, ...), so a comparison is needed at every step but the last

Merge sort: dividing



- Keep dividing until each sub-list has just one element (the base case)
- A single element is trivially **sorted**

Merge sort: merging back up



Each pair of sorted lists is **merged** back together, in sorted order, all the way up.

Merge sort complexity: work per level

For a list of size 8:

- 4 merges of two lists of size 1
- 2 merges of two lists of size 2
- 1 merge of two lists of size 4

In general, at each level:

- $n/2$ merges of two lists of size 1: complexity $= n/2 \times 2$
- $n/4$ merges of two lists of size 2: complexity $= n/4 \times 4$
- ...
- 1 merge of two lists of size $n/2$: complexity $= n$

Each level costs $O(n)$, no matter how many merges make it up

Merge sort complexity

Merge sort complexity = work per level × number of levels

- Number of levels $O(\log n)$ (like binary search)
- Log-linear: $O(n \log n)$
- Big improvement over selection sort!
- Does need some more space due to copying lists

Complexity classes

Fast algorithm: worst-case running time grows slowly with input size

- $O(1)$: constant running time (primitive operations)
- $O(\log n)$: logarithmic running time (binary search)
- $O(n)$: linear running time (linear search)
- $O(n \log n)$: log-linear time (merge sort)
- $O(n^c)$: polynomial running time (selection sort)
- $O(c^n)$: exponential running time (??)

Sorting is a canonical algorithms problem

Many algorithms exist: bubble sort, insertion sort, quick sort, heap sort, ...

- Useful for developing algorithmic thinking (e.g. randomized algorithms)

Theoretical bound for worst-case performance is $O(n \log n)$: we can't do better than merge sort

But other algorithms **can be faster** in practice:

- Python uses **timsort**, created by Tim Peters in 2002 out of frustration with existing algorithms
- Exploits the fact that lists tend to be partly sorted already
- Sorting research is still active: a 2018 paper introduced **powersort**, an improved merge strategy that Python adopted in version 3.11

Try it yourself

Sorting algorithm visualizer: sorting-algorithms-app-v2.vercel.app

Review

Sorting is a canonical computer science problem:

- We've looked at two (of many) algorithms
- Selection sort involves repeatedly finding the minimum element (intuitive but slow)
- Merge sort is faster and has a neat recursive structure

Review exercises:

- Implement sorting
- More looping and function practice
- Challenges: implement a recursive version of binary search and an iterative merge sort

Why can't sorting beat $n \log n$? (optional)

Think of any comparison-based sorting algorithm as a **decision tree**:

- Each comparison is a branch, with 2 possible outcomes
- Each leaf is one final ordering of the input

For n distinct items, there are $n!$ possible orderings, so the tree needs at least $n!$ leaves, one for each possible correct answer.

A tree of depth d has at most 2^d leaves, so we need:

$$2^d \geq n! \qquad \Rightarrow \qquad d \geq \log_2(n!)$$

comparisons, in the worst case.

From $\log_2(n!)$ to $n \log n$ (optional)

$$\log_2(n!) = \log_2 n + \log_2(n-1) + \cdots + \log_2 1$$

Just the top half of these n terms are each at least $\log_2(n/2)$:

$$\log_2(n!) \geq \frac{n}{2} \times \log_2(n/2) = \Omega(n \log n)$$

Comparison sorts can't beat $O(n \log n)$ (optional)

In the worst case:

- Merge sort isn't just fast, it's asymptotically optimal among comparison-based sorts
- This bound is specific to comparison sorts: methods like counting sort or radix sort don't compare elements, and can beat it
- These are used in practice, but only for restricted kinds of data: counting sort for integers within a known range, radix sort for fixed-width keys like fixed-length strings or IP addresses